app Operations

Overview

This document explains, at a high level, how each service in the app service stack is used for the app. It is intended as an operations guide, not a technical implementation spec.

The current stack in the PDF covers these areas:

- domain and DNS management
- website hosting and SSL
- payment processing
- subscription and app monetization management
- marketing email
- transactional email for coupons, logins, and app events
- internal business inboxes
- SMS delivery for mobile verification and OTP

1. Cloudflare Registrar

Primary role: Own and renew the app domain.

How app should use it:

- Keep the main production domain registered in Cloudflare.
- Use Cloudflare as the central place for DNS records that point the website, email services, and app-related subdomains to the right providers.
- Restrict domain-level access to a very small number of operators.

Typical operating tasks:

- renew the domain before expiration
- update DNS records when a service is added or replaced
- verify nameservers and domain lock settings
- keep billing details current

High-level note:

This is infrastructure control, not an app feature. If the domain or DNS is misconfigured, the website, email, and API-related endpoints can all be affected.

Headless REST API Example (Where Plausible)

Cloudflare Registrar is mostly an operations console concern rather than an in-app runtime API. In a headless setup, app would usually not call registrar APIs directly from the app.

Plausible automation pattern:

1. Ops creates a DNS change request (for example, adding `api.app.com`).

2. CI or infra scripts apply DNS changes through Cloudflare APIs.
3. The app backend health endpoint verifies the new host is reachable before rollout.

2. Cloudflare Pages

Primary role: Host the app website with SSL.

How app should use it:

- Deploy the public website and landing pages through Cloudflare Pages.
- Use it for fast static hosting, SSL certificates, and previews for content or design updates.
- Connect the production domain so the site is always served over HTTPS.

Typical operating tasks:

- deploy site updates from the repository
- review preview builds before publishing
- attach custom domains and validate SSL
- monitor build usage and move to a paid tier if build limits or advanced security controls are needed

High-level note:

This service is for the web presence around app. It should stay separate from app messaging, subscription logic, and transactional communications.

Headless REST API Example

Cloudflare Pages hosts the frontend that consumes your headless API.

Example flow:

1. Website calls app API: `GET /api/v1/coupons/featured`.
2. API responds with JSON list of featured offers.
3. Frontend renders coupons and links users into app sign-in/claim flow.

Example response shape:

```
{
    "items": [
        {
            "couponId": "cpn_123",
            "title": "20% off brunch",
            "expiresAt": "2026-04-15T23:59:59Z"
        }
    ]
}
```

3. Stripe

Primary role: Process payments and act as the financial billing layer.

How app should use it:

- Use Stripe for any direct online payments tied to app, especially if app offers paid plans, recurring billing, or web-based checkout flows.
- Treat Stripe as the source of truth for charges, refunds, invoices, and payout reporting.
- If subscriptions are sold outside the mobile app stores, Stripe should handle that recurring billing logic.

Typical operating tasks:

- create products and prices
- review successful and failed payments
- issue refunds when needed
- monitor subscription billing behavior and fees
- reconcile finance reports with app analytics and subscription events

High-level note:

Stripe is about money movement and reporting. It should be operated carefully with clear ownership across product, finance, and support.

Headless REST API Example

If app supports web checkout, the backend creates Stripe Checkout sessions from a headless endpoint.

Example flow:

1. Client calls `POST /api/v1/billing/checkout-session` with selected plan.
2. app backend creates a Stripe checkout session server-side.
3. Backend returns `checkoutUrl`.
4. Client redirects user to Stripe checkout.

Example request:

```
{
    "planCode": "pro_monthly",
    "userId": "usr_42"
}
```

Example response:

```
{
    "checkoutUrl": "https://checkout.stripe.com/c/pay/cs_test_..."
}
```

4. Adapty

Primary role: Manage app subscriptions, entitlements, and monetization analytics.

How app should use it:

- Use Adapty as the subscription management layer for the mobile app.
- Connect it to App Store and Google Play subscription products so app can control access to premium features.
- Use Adapty to manage paywalls, entitlement logic, and revenue tracking.

Typical operating tasks:

- connect store products and map them to app plans
- define which features unlock for each entitlement
- monitor monthly tracked revenue against the free-tier threshold
- review conversion and churn trends
- test paywalls and pricing changes before rollout

High-level note:

Adapty is the operational control panel for app monetization. It is not the primary payment gateway itself, but it helps app manage subscription behavior across app stores.

Headless REST API Example

The app sends purchase/subscription state to app API, and the backend syncs entitlement state using Adapty data.

Example flow:

1. Mobile app completes in-app purchase.
2. App calls `POST /api/v1/subscriptions/sync` with store receipt/token.
3. app backend validates through Adapty integration.
4. Backend updates user entitlement and returns access state.

Example response:

```
{
    "userId": "usr_42",
    "entitlements": ["premium"],
    "isActive": true,
    "renewsAt": "2026-04-14T10:00:00Z"
}
```

5. Audienceful

Primary role: Run marketing email campaigns and lifecycle email sequences.

How app should use it:

- Use Audienceful for newsletters, product announcements, onboarding drips, and promotional campaigns.
- Segment users by lifecycle stage, geography, or engagement level.
- Keep marketing email separate from transactional login and OTP email.

Typical operating tasks:

- maintain subscriber lists and segments
- create campaign emails and automated sequences
- monitor contact count against the free-tier limit
- authenticate the sending domain
- review open, click, and unsubscribe trends

High-level note:

Audienceful should be treated as the outbound marketing channel for growth and retention, not for critical account access flows.

Headless REST API Example (Where Plausible)

Marketing email is often event-driven from backend user actions.

Example flow:

1. User completes onboarding.
2. app backend emits event `user.onboarding_completed`.
3. Event worker adds/updates contact in Audienceful list/segment.
4. Audienceful sends lifecycle campaign.

Example internal event payload:

```
{
    "event": "user.onboarding_completed",
    "userId": "usr_42",
    "email": "user@example.com",
    "country": "GR"
}
```

6. Resend

Primary role: Send transactional email for the app.

How app should use it:

- Use Resend for app-generated emails such as coupon issued notifications, login links, verification emails, welcome messages, and important account notifications.
- Keep these messages fast, reliable, and operationally separate from marketing campaigns.
- Use the service for API-triggered email events from the app or backend workflows.

Typical operating tasks:

- trigger a coupon email whenever a coupon is issued to a user
- configure the sending domain and authentication records
- create templates for login and account emails
- create and maintain coupon email templates
- monitor daily and monthly send limits
- review bounce and delivery behavior
- upgrade once usage exceeds the free-tier daily ceiling or monthly volume

High-level note:

Resend supports core user flows. If it fails, users may be unable to sign in or complete important account actions.

Headless REST API Example

Coupon issuance should trigger an email dispatch through app backend.

Example flow:

1. Admin or automation issues coupon via `POST /api/v1/coupons/issue`.
2. Backend stores coupon assignment.
3. Backend calls Resend API with coupon template.
4. API returns success with coupon and email delivery status.

Example request:

```
{
    "userId": "usr_42",
    "couponTemplateId": "tmpl_welcome_20"
}
```

Example response:

```
{
    "couponId": "cpn_123",
    "email": {
        "provider": "resend",
        "status": "queued"
    }
}
```

7. Zoho Mail

Primary role: Provide internal business inboxes.

How app should use it:

- Use Zoho Mail for human-managed inboxes such as support, operations, partnerships, and admin.

- Keep it as the team communication layer for messages that require manual handling.
- Use role-based inboxes where possible instead of personal email for business continuity.

Typical operating tasks:

- create and manage staff mailboxes
- enable mobile access if the team upgrades beyond web-only access
- maintain forwarding rules and shared inbox workflows
- keep account recovery and security settings current

High-level note:

Zoho Mail is for people, not automated app delivery. It should not replace Resend for transactional email.

Headless REST API Example (Where Plausible)

Zoho Mail is usually for manual team operations, but you can connect support workflows.

Example flow:

1. User submits issue to `POST /api/v1/support/tickets`.
2. Backend stores ticket and notifies support mailbox (for example, `support@app.com`).
3. Support responds manually via Zoho mailbox.

Example response:

```
{
    "ticketId": "tkt_9001",
    "status": "open"
}
```

8. Twilio

Primary role: Deliver SMS for mobile phone verification and OTP.

How app should use it:

- Use Twilio for one-time passwords, mobile number verification, and authentication-related SMS flows.
- Keep Twilio focused on phone-based security and avoid mixing it with email use cases.
- Add notification SMS only when needed for urgent user communication.

Typical operating tasks:

- configure sender identity, routing, and verification settings
- monitor OTP delivery success rates

- track per-message cost and fee impact
- keep templates short and operational

High-level note:

This is a fallback or primary auth channel depending on product design. It should be closely monitored because failed OTP delivery directly blocks user access.

Headless REST API Example

Mobile verification and OTP are ideal for a REST-first auth flow.

Example flow:

1. App starts verification: `POST /api/v1/auth/phone/start` with phone number.
2. Backend asks Twilio to send OTP SMS.
3. App verifies code: `POST /api/v1/auth/phone/verify`.
4. Backend validates code and issues session/JWT.

Example start request:

```
{
    "phone": "+3069XXXXXXXX"
}
```

Example verify request:

```
{
    "phone": "+3069XXXXXXXX",
    "otp": "123456"
}
```

Example verify response:

```
{
    "verified": true,
    "accessToken": "eyJhbGciOi..."
}
```

Recommended Service Boundaries

To keep operations clean, app should use each service for a distinct job:

- Cloudflare Registrar: domain ownership and DNS
- Cloudflare Pages: website hosting and SSL
- Stripe: direct payment processing and billing records
- Adapty: subscription orchestration and app monetization analytics
- Audienceful: marketing email
- Resend: transactional app email, including coupon issued emails

- Zoho Mail: internal team inboxes
- Twilio: SMS mobile verification and OTP

Basic Operating Rhythm

At a high level, app should review this stack on a regular cadence:

- weekly: check app sign-in delivery, payment failures, and website uptime
- monthly: review spend, plan thresholds, email list growth, and subscription metrics
- quarterly: review whether free tiers are still sufficient and whether any tool should be consolidated or upgraded

Risks To Watch

- DNS mistakes can affect the whole stack.
- Mixing marketing and transactional email can hurt deliverability.
- OTP channels must be monitored because authentication depends on them.
- Subscription logic should be kept aligned across Adapty, Stripe, and the app stores.
- Coupon email delivery should be monitored so users reliably receive issued offers.

Summary

The app service stack is straightforward if each tool keeps a single responsibility. Cloudflare handles the domain and website, Stripe and Adapty handle revenue flows, Audienceful and Resend handle different kinds of email, Zoho supports human inboxes, and Twilio covers phone verification and OTP messaging.